

Lista de Código: Projeto Integrado (Full Stack)

Turma: LionsDev

Tópicos: integração de todo o curso: modelagem (Mongoose), API REST (Express), autenticação (bcrypt + JWT), organização em camadas, deploy (variáveis de ambiente e Render) e frontend consumindo a API.

Cada exercício constrói uma camada de uma aplicação completa e a conecta à anterior. Ao final da Parte 4, você terá um projeto full stack de ponta a ponta. Reaproveite o que foi visto nos módulos 06 a 11.

Parte 0: Exercícios de fixação

Itens curtos, um de cada camada. Resolva cada um isoladamente e identifique de qual módulo vem o conteúdo.

1. **Model:** escreva um `Schema` de `Tarefa` com `titulo` (String, obrigatório) e `concluida` (Boolean, padrão `false`).
2. **Rota:** escreva a rota `GET /tarefas` que lista do banco com `await Tarefa.find()`.
3. **Create:** escreva `POST /tarefas` que cria com `await Tarefa.create(req.body)` e responde 201.
4. **Auth:** gere o hash da senha no cadastro com `bcrypt.hash`.
5. **Token:** no login, assine um token com `jwt.sign({ id }, process.env.JWT_SECRET)`.
6. **Middleware:** proteja `GET /perfil` exigindo `Authorization: Bearer`.
7. **Env:** faça a porta ser `process.env.PORT || 3000`.
8. **Camadas:** diga em qual camada (model, repository, service, controller ou routes) fica a regra "gerar o hash".
9. **Frontend:** escreva o prompt de uma tela que lista `GET /tarefas` em cards.
10. **Deploy:** liste as variáveis de ambiente que precisam ser configuradas no painel do Render (`MONGO_URI`, `JWT_SECRET`) e explique por que `PORT` não precisa ser configurada.

Parte 1: Complete a integração

1. Ligando controller e service

Complete o controller para chamar o service e responder.

```
// service
async function listarTarefas() {
  return await Tarefa.find();
}

// controller
async function getTarefas(req, res) {
  const tarefas = /* TODO: await chamar o service */;
  // TODO: responda 200 com as tarefas
}
```

2. Rota protegida

Complete a definição da rota aplicando o middleware de autenticação.

```
// só usuários logados podem ver o perfil
router.get("/perfil", /* TODO: middleware */, perfilController);
```

Parte 2: Ache o problema de integração

3. O frontend não recebe nada

A tela chama `GET /tarefas` e sempre recebe `[]`, mesmo com dados no banco. Liste 3 causas prováveis olhando a cadeia inteira (o frontend aponta para a API certa, local ou Render? A `MONGO_URI` dessa API aponta para o mesmo banco onde estão os dados? O repository filtra por `req.usuario.id` e os registros pertencem a outro usuário?).

4. Rota protegida barra todo mundo

Depois de logar, o usuário chama `GET /perfil` e recebe 401 mesmo com o token. Aponte onde investigar (o frontend está enviando `Authorization: Bearer TOKEN`? O segredo do `verify` é o mesmo do `sign`? O token expirou?).

Parte 3: Prever o fluxo ponta a ponta

5. A jornada de uma requisição

Descreva, em ordem, por quais camadas passa uma requisição de cadastro de tarefa, do clique no frontend até a resposta:

```
Frontend (fetch POST /tarefas)
-> ? (o que recebe primeiro no backend)
-> ? (quem valida o token)
-> ? (quem lê req.body e chama a lógica)
-> ? (quem aplica a regra de negócio)
-> ? (quem acessa o banco)
-> ? (o que volta para o frontend)
```

Preencha cada `?` com a camada/responsável correto.

Parte 4: Construa o Projeto Final

6. Aplicação Full Stack (Desafio final)

Escolha um tema (agenda, finanças, petshop, biblioteca...) e construa a aplicação completa, integrando todos os módulos. Os requisitos mínimos estão em [projeto_final.md](#).

1. **Requisitos:** liste as entidades e as rotas (o que o sistema faz).
2. **Model:** schemas no Mongoose com validações.
3. **API:** CRUD completo em Express, organizado em camadas (model, repository, service, controller e routes).
4. **Auth:** cadastro/login com bcrypt + JWT; rotas privadas protegidas por middleware.
5. **Config:** porta, URI e segredo via `process.env`; `.env` no `.gitignore`; script `start`.
6. **Deploy:** publique a API no Render com as variáveis configuradas.
7. **Frontend:** gere as telas (IA/FlutterFlow) consumindo a API publicada.
8. **Teste ponta a ponta:** cadastrar usuário → login → criar/listar/editar/deletar dados pela tela.

Entregue: repositório no GitHub, links do backend e do frontend publicados e um roteiro de teste mostrando o fluxo completo funcionando.

Dica: em um projeto full stack, a requisição passa por frontend, rota, middleware, controller, service, repository, model e banco, e a resposta volta pelo mesmo caminho.

Quando algo não funciona, siga esse caminho com um `console.log` em cada camada até encontrar o ponto em que o dado se perde.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: Projeto Integrado Full Stack - Módulo 12